Webhooks

Ask AI

Webhooks allow your platform to receive real-time notifications from Stream when key events occur—such as payment successes, failures, or cancellations. This enables you to react to events programmatically, such as updating your database, sending notifications, or triggering internal workflows.

Stream's webhook infrastructure is designed for reliability and includes features like:

- Event-specific subscriptions
- Secure payload signing
- Delivery tracking for observability
- Automatic retries using exponential backoff

Creating a Webhook

To create or manage your webhooks, go to Webhook Settings.

There you can:

- Register a new webhook endpoint
- Select which event types you want to subscribe to
- Enable, disable, or delete webhooks

Webhook Flow Logic

Understanding which webhooks are sent in different scenarios:

Single Payment Succeeds

When a single payment succeeds:

1. `PAYMENT_SUCCEEDED` webhook is sent (one per payment)
2. If the payment completes the invoice (all payments are now paid), `INVOICE_COMPLETED` webhook is also sent

Example: Invoice with 1 payment of 1000 SAR

- Payment succeeds → `PAYMENT_SUCCEEDED` webhook
- Invoice completes → `INVOICE_COMPLETED` webhook

Multiple Payments Succeed (Installments)

When multiple payments succeed together (e.g., installments):

1. `PAYMENT_SUCCEEDED` webhook is sent for each payment that succeeded (one per payment)
2. If all payments complete the invoice, `INVOICE_COMPLETED` webhook is also sent

Example: Invoice with 3 payments of 1000 SAR each

- Payments 1, 2, 3 all succeed together:
  - `PAYMENT_SUCCEEDED` webhook for payment_1
  - `PAYMENT_SUCCEEDED` webhook for payment_2
  - `PAYMENT_SUCCEEDED` webhook for payment_3
  - `INVOICE_COMPLETED` webhook

Example: Invoice with 5 payments, payments 1-3 succeed together:

- `PAYMENT_SUCCEEDED` webhook for payment_1
- `PAYMENT_SUCCEEDED` webhook for payment_2
- `PAYMENT_SUCCEEDED` webhook for payment_3
- No `INVOICE_COMPLETED` webhook (invoice not yet fully paid)

Payment Fails

When a payment fails:

1. `PAYMENT_FAILED` webhook is sent
2. No invoice completion webhook is sent

Payment Refunded

When a payment is refunded:

1. `PAYMENT_REFUNDED` webhook is sent
2. Invoice status does not change (invoice remains completed)

Payment Marked as Paid

When a payment is manually marked as paid:

1. `PAYMENT_MARKED_AS_PAID` webhook is sent
2. If this completes the invoice, `INVOICE_COMPLETED` webhook is also sent

Subscription Renewal

When a subscription cycle renews:

1. New invoice created → `INVOICE_CREATED` webhook
2. Payment succeeds → `PAYMENT_SUCCEEDED` webhook
3. Invoice completes → `INVOICE_COMPLETED` webhook

Note: For subscription renewals, the `INVOICE_COMPLETED` webhook indicates that the subscription cycle has been renewed successfully. The invoice will have a `subscription_id` in its payload, and you can check if it's a renewal invoice by comparing it with previous invoices for that subscription.

Event Types

Payment Events

- `PAYMENT_SUCCEEDED` - Payment completed successfully
- `PAYMENT_FAILED` - Payment attempt failed
- `PAYMENT_CANCELED` - Payment was canceled
- `PAYMENT_REFUNDED` - Payment was refunded
- `PAYMENT_MARKED_AS_PAID` - Payment was manually marked as paid

Invoice Events

- `INVOICE_CREATED` - New invoice created
- `INVOICE_SENT` - Invoice sent to consumer
- `INVOICE_ACCEPTED` - Invoice accepted by consumer
- `INVOICE_REJECTED` - Invoice rejected by consumer
- `INVOICE_COMPLETED` - Invoice completed (all payments done)
- `INVOICE_CANCELED` - Invoice canceled
- `INVOICE_UPDATED` - Invoice details updated

Subscription Events

- `SUBSCRIPTION_CREATED` - New subscription created
- `SUBSCRIPTION_ACTIVATED` - Subscription activated
- `SUBSCRIPTION_INACTIVATED` - Subscription deactivated
- `SUBSCRIPTION_CANCELED` - Subscription canceled
- `SUBSCRIPTION_FROZEN` - Subscription frozen
- `SUBSCRIPTION_CYCLE_RENEWAL_FAILED` - Subscription cycle renewal failed (use `INVOICE_COMPLETED` with `subscription_id` for successful renewals)
- `SUBSCRIPTION_CANCEL_AT_PERIOD_END` - Subscription scheduled to cancel at period end
- `SUBSCRIPTION_FREEZE_NOW` - Subscription frozen immediately
- `SUBSCRIPTION_UNFREEZE_NOW` - Subscription unfrozen immediately
- `SUBSCRIPTION_UNFREEZE_FUTURE` - Subscription scheduled to unfreeze in future
- `SUBSCRIPTION_FREEZE_CANCEL` - Subscription freeze canceled

Payment Link Events

- `PAYMENT_LINK_PAY_ATTEMPT_FAILED` - Payment attempt failed for a payment link

  Note: This webhook is sent when a payment link payment fails. Unlike regular payment failures, no invoice, payment, or subscription entities are created on failure, so `PAYMENT_FAILED` webhook cannot be sent. This event exists specifically to notify about payment link failures when no entities exist.

Webhook Payload

The webhook POST request will have a JSON body like:

```
{
  "event_type": "PAYMENT_SUCCEEDED",
  "entity_type": "PAYMENT",
  "entity_id": "uuid",
  "entity_url": "<https://stream-app-service.streampay.sa/api/v2/payments/{id}>",
  "status": "SUCCEEDED",
  "data": { /* payment or event-specific data */ },
  "timestamp": "2025-07-15T14:41:21.705Z"
}
```

Webhook Payload Example

```
{
  "data": {
    "invoice": {
      "id": "2df0f7e0-2634-46ab-829a-bfcb0a797d87",
      "url": "https://stream-app-service.streampay.sa/api/v2/invoices/2df0f7e0-
2634-46ab-829a-bfcb0a797d87"
    },
    "payment": {
      "id": "e2182d3d-b4cf-4972-bcc0-ec6d963c066d",
      "url": "https://stream-app-service.streampay.sa/api/v2/payments/e2182d3d-
b4cf-4972-bcc0-ec6d963c066d"
    },
    "payment_link": {
      "id": "6361941e-3a81-4aa5-aaa6-60d6e052883a",
      "url": "https://stream-app-
service.streampay.sa/api/v2/payment_links/6361941e-3a81-4aa5-aaa6-60d6e052883a"
    },
    "metadata": {
      "customer_id": "cust_12345",
      "nested_data": {
        "sub_field": "sub_value",
        "sub_number": 678
```

```
      },
      "affiliate_id": "aff_987",
      "custom_field_1": "value_1",
      "custom_field_2": 12345,
      "campaign_source": "email_marketing"
    }
  },
  "status": "SUCCEEDED",
  "entity_id": "e2182d3d-b4cf-4972-bcc0-ec6d963c066d",
  "timestamp": "2025-07-22T14:40:31.485576",
  "entity_url": "https://stream-app-
service.streampay.sa/api/v2/payments/e2182d3d-b4cf-4972-bcc0-ec6d963c066d",
  "event_type": "PAYMENT_SUCCEEDED",
  "entity_type": "PAYMENT"
}
```

Webhook Request Headers

- `Content-Type: application/json`
- `User-Agent: StreamApp-Webhook/1.0`
- `X-Webhook-Event`: Event type (e.g., `PAYMENT_SUCCEEDED`)
- `X-Webhook-Entity-Type`: Entity type (e.g., `PAYMENT`)
- `X-Webhook-Entity-ID`: Entity UUID
- `X-Webhook-Signature`: Format: `t={timestamp},v1={signature}`
  - `timestamp` is the UNIX timestamp when the signature was generated.
  - `signature` is an HMAC-SHA256 of `{timestamp}.{payload}` using your webhook's `secret_key`.
- `X-Webhook-Timestamp`: The timestamp used in the signature.

Verifying Webhook Authenticity

To verify a webhook:

1. Extract the `X-Webhook-Signature` header and split into timestamp and signature.

2. Compute the HMAC-SHA256 of `{timestamp}.{raw_request_body}` using your webhook's `secret_key`.
3. Compare your computed signature to the value in the header.

Example (Python):

```
import hmac, hashlib

def verify_webhook_signature(secret: str, raw_body: bytes, signature_header:
str):
    # signature_header: "t=TIMESTAMP,v1=SIGNATURE"
    parts = dict(x.split('=') for x in signature_header.split(','))
    timestamp = parts['t']
    signature = parts['v1']
    message = f"{timestamp}.{raw_body.decode()}"
    computed = hmac.new(secret.encode(), message.encode(),
hashlib.sha256).hexdigest()
    return hmac.compare_digest(computed, signature)
```

Webhook Delivery & Retry Logic

- Delivery Attempts:

  Every webhook event triggers a delivery record. Each delivery attempt and its status (pending, delivered, failed, retrying) is tracked for auditing.

- Retry Schedule:

  If your endpoint does not respond with a 2xx status, Stream will retry delivery up to 5 times.

  The default retry delays (in minutes) are:

  i. 1st retry: 5 minutes after the first failure
  ii. 2nd retry: 30 minutes after the second failure
  iii. 3rd retry: 2 hours (120 minutes) after the third failure
  iv. 4th retry: 6 hours (360 minutes) after the fourth failure
  v. 5th retry: 12 hours (720 minutes) after the fifth failure

After the fifth attempt, if delivery is still unsuccessful, the webhook delivery is marked as failed and no further attempts will be made.

- Auditing:

  All delivery attempts, responses, and errors are logged and can be audited.